阿里云天池 Python 训练营-Part7

Excel 中的常用函数

Excel 中的常用函数非常多，按照功能类别可以分为以下几大类，Excel 最新版本（如 Microsoft 365 和 Excel 2021）中支持的函数总数超过 **500 个**。以下是常见功能类别及一些常用的代表性函数：

---

1. 数学与三角函数

用于执行数学和三角运算。

- **常用函数**：
  - `SUM`：求和。
  - `AVERAGE`：计算平均值。
  - `ROUND`：四舍五入。
  - `INT`：取整。
  - `MOD`：取余数。
  - `RAND`：生成随机数。
  - `PI`：返回圆周率。
  - `SIN` / `COS` / `TAN`：三角函数。

---

2. 统计函数

用于执行统计分析。

- **常用函数**：
  - `COUNT`：统计单元格个数。
  - `COUNTA`：统计非空单元格个数。
  - `COUNTIF`：按条件统计单元格。
  - `MAX` / `MIN`：返回最大值/最小值。
  - `MEDIAN`：计算中位数。
  - `STDEV.P` / `STDEV.S`：计算标准偏差。
  - `VAR.P` / `VAR.S`：计算方差。

---

3. 文本函数

用于操作和处理文本。

- **常用函数**：
  - `TEXT`：格式化文本。
  - `LEFT` / `RIGHT`：提取文本左/右部分。
  - `MID`：提取文本中间部分。
  - `LEN`：计算文本长度。
  - `TRIM`：去除多余空格。
  - `CONCAT` / `TEXTJOIN`：合并文本。
  - `SEARCH` / `FIND`：查找文本位置。

---

4. 日期和时间函数

用于操作日期和时间数据。

- **常用函数**：
  - `TODAY`：返回当前日期。
  - `NOW`：返回当前日期和时间。
  - `DATE`：返回日期值。
  - `YEAR` / `MONTH` / `DAY`：提取年、月、日。
  - `HOUR` / `MINUTE` / `SECOND`：提取时、分、秒。
  - `DATEDIF`：计算日期差值。
  - `WEEKDAY`：返回星期几。

---

5. 查找与引用函数

用于查找数据或引用其他单元格。

- **常用函数**：
  - `VLOOKUP`：垂直查找。
  - `HLOOKUP`：水平查找。
  - `INDEX`：返回单元格引用。
  - `MATCH`：返回匹配值的位置。
  - `XLOOKUP`：功能更强大的查找函数。
  - `OFFSET`：返回偏移引用。

6. 逻辑函数

用于执行逻辑测试。

- **常用函数**：
  - `IF`：逻辑判断。
  - `AND`：多个条件同时为真。
  - `OR`：任一条件为真。
  - `NOT`：逻辑非。
  - `IFERROR`：处理错误值。

---

7. 财务函数

用于财务分析和计算。

- **常用函数**：
  - `PV`：计算现值。
  - `FV`：计算终值。
  - `PMT`：计算分期付款额。
  - `NPER`：计算支付期数。
  - `RATE`：计算利率。

---

8. 信息函数

用于返回有关单元格、公式的信息。

- **常用函数**：
  - `ISBLANK`：检查是否为空。
  - `ISERROR`：检查是否有错误。
  - `TYPE`：返回值的数据类型。
  - `INFO`：返回关于工作环境的信息。

---

9. 数组函数（动态数组）

动态数组在新版 Excel 中引入，允许公式返回多个值。

- **常用函数**：
  - `FILTER`：筛选数组。
  - `UNIQUE`：返回唯一值。
  - `SORT`：对数组排序。
  - `SEQUENCE`：生成序列。
  - `ARRAYFORMULA`：对数组操作。

---

10. 工程函数

用于执行工程计算。

- **常用函数**：
  - `CONVERT`：单位转换。
  - `DEC2BIN`：十进制转二进制。
  - `HEX2DEC`：十六进制转十进制。

常见问题

你可以通过 Excel 内置的“**插入函数**”窗口查看所有函数并按类别筛选，或者直接通过函数库快速找到对应功能的函数。

1. Python 实现数学与三角函数

函数列表：`SUM`、`AVERAGE`、`ROUND`、`INT`、`MOD`、`RAND`、`PI`、`SIN`、`COS`、`TAN`

1.1 `SUM`

- **描述**：计算一组数值的总和。
- **Excel 示例**：

A	B
10	=SUM(A1:A3)
20	
30	

- **Excel 解决方案**：`=SUM(A1:A3)`

- **Python 实现**：

```python
def sum_values(values):
    return sum(values)

print(sum_values([10, 20, 30]))
```

1.2 `AVERAGE`

- **描述**：计算一组数值的平均值。
- **Excel 示例**：

A	B
85	=AVERAGE(A1:A3)
90	
95	

- **Excel 解决方案**：`=AVERAGE(A1:A3)`
- **Python 实现**：

```python
def average(values):
    return sum(values) / len(values) if values else 0

print(average([85, 90, 95]))
```

1.3 `ROUND`

- **描述**：将数值四舍五入到指定的小数位数。
- **Excel 示例**：

A	B
3.14159	=ROUND(A1, 2)

- **Excel 解决方案**：`=ROUND(A1, 2)`
- **Python 实现**：

```python
def round_value(value, digits):
    return round(value, digits)

print(round_value(3.14159, 2))
```

1.4 `INT`

- **描述**：将数值向下取整。
- **Excel 示例**：

A	B
4.7	=INT(A1)

- **Excel 解决方案**：`=INT(A1)`
- **Python 实现**：

```python
def int_value(value):
    return int(value)

print(int_value(4.7))
```

1.5 `MOD`

- **描述**：返回两数相除后的余数。
- **Excel 示例**：

A	B	C
10	3	=MOD(A1,B1)

- **Excel 解决方案**：`=MOD(A1, B1)`
- **Python 实现**：

```python
def mod_value(a, b):
    return a % b

print(mod_value(10, 3))
```

1.6 `RAND`

- **描述**：生成一个介于 0 和 1 之间的随机数。
- **Excel 示例**：

A
=RAND()

- **Excel 解决方案**：`=RAND()`
- **Python 实现**：

```python
import random

def rand_value():
    return random.random()

print(rand_value())
```

1.7 `PI`

- **描述**：返回圆周率 π 的值。
- **Excel 示例**：

A
=PI()

- **Excel 解决方案**：`=PI()`
- **Python 实现**：

```python
import math

def pi_value():
    return math.pi

print(pi_value())
```

1.8 `SIN` / `COS` / `TAN`

- **描述**：计算角度的正弦、余弦或正切值。
- **Excel 示例**：

A	B
30	=SIN(A1)
45	=COS(A2)
60	=TAN(A3)

- **Excel 解决方案**：
  - `=SIN(A1)`

- `=COS(A2)`
- `=TAN(A3)`

- **Python 实现**：

```python
def sin_value(angle):
    return math.sin(math.radians(angle))

def cos_value(angle):
    return math.cos(math.radians(angle))

def tan_value(angle):
    return math.tan(math.radians(angle))

print(sin_value(30))  # 正弦
print(cos_value(45))  # 余弦
print(tan_value(60))  # 正切
```

下面继续实现其余 **9 大类函数**中的所有函数，按照之前的结构逐一列出。

---

2. Python 实现统计函数

函数列表： `COUNT`、`COUNTA`、`COUNTIF`、`MAX`、`MIN`、`MEDIAN`、`STDEV.P`、`STDEV.S`、`VAR.P`、`VAR.S`

2.1 `COUNT`

- **描述**：统计范围内数值单元格的数量。
- **Excel 示例**：

A	B
10	=COUNT(A1:A3)
20	
空白	

- **Excel 解决方案**：`=COUNT(A1:A3)`
- **Python 实现**：

```python
def count_values(values):
```

```
    return sum(1 for v in values if isinstance(v, (int, float)))

print(count_values([10, 20, None]))
```

2.2 `COUNTA`

- **描述**：统计范围内非空单元格的数量。
- **Excel 示例**：

A	B
10	=COUNTA(A1:A3)
空白	
文本	

- **Excel 解决方案**：`=COUNTA(A1:A3)`
- **Python 实现**：

```
def counta_values(values):
    return sum(1 for v in values if v is not None and v != "")

print(counta_values([10, "", "文本"]))
```

2.3 `COUNTIF`

- **描述**：统计范围内符合条件的单元格数量。
- **Excel 示例**：

A	B
10	=COUNTIF(A1:A3, ">15")
20	
30	

- **Excel 解决方案**：`=COUNTIF(A1:A3, ">15")`
- **Python 实现**：

```
def countif_values(values, condition):
    operator, threshold = condition[0], float(condition[1:])
    if operator == ">":
```

```
        return sum(1 for v in values if v > threshold)
    elif operator == "<":
        return sum(1 for v in values if v < threshold)
    elif operator == "=":
        return sum(1 for v in values if v == threshold)
    else:
        raise ValueError("不支持的条件操作符")

print(countif_values([10, 20, 30], ">15"))
```

2.4 `MAX`

- **描述**：返回范围中的最大值。
- **Excel 示例**：

A	B
10	=MAX(A1:A3)
20	
30	

- **Excel 解决方案**：`=MAX(A1:A3)`
- **Python 实现**：

```
def max_value(values):
    return max(values)

print(max_value([10, 20, 30]))
```

2.5 `MIN`

- **描述**：返回范围中的最小值。
- **Excel 示例**：

A	B
10	=MIN(A1:A3)
20	
30	

- **Excel 解决方案**：`=MIN(A1:A3)`

- **Python 实现**：

```python
def min_value(values):
    return min(values)

print(min_value([10, 20, 30]))
```

2.6 `MEDIAN`

- **描述**：计算一组数值的中位数。
- **Excel 示例**：

A	B
10	=MEDIAN(A1:A3)
20	
30	

- **Excel 解决方案**：`=MEDIAN(A1:A3)`
- **Python 实现**：

```python
def median_value(values):
    sorted_values = sorted(values)
    n = len(sorted_values)
    if n % 2 == 0:
        return (sorted_values[n // 2 - 1] + sorted_values[n // 2]) / 2
    else:
        return sorted_values[n // 2]

print(median_value([10, 20, 30]))
```

2.7 `STDEV.P` & `STDEV.S`

- **描述**：计算总体（P）或样本（S）的标准差。
- **Excel 示例**：

A	B
10	=STDEV.P(A1:A3)
20	=STDEV.S(A1:A3)
30	

- **Excel 解决方案**：
  - 总体标准差：`=STDEV.P(A1:A3)`
  - 样本标准差：`=STDEV.S(A1:A3)`
- **Python 实现**：

```python
import statistics

def stdev_p(values):
    return statistics.pstdev(values)

def stdev_s(values):
    return statistics.stdev(values)

print(stdev_p([10, 20, 30]))
print(stdev_s([10, 20, 30]))
```

2.8 `VAR.P` & `VAR.S`

- **描述**：计算总体（P）或样本（S）的方差。
- **Excel 示例**：

A	B
10	=VAR.P(A1:A3)
20	=VAR.S(A1:A3)
30	

- **Excel 解决方案**：
  - 总体方差：`=VAR.P(A1:A3)`
  - 样本方差：`=VAR.S(A1:A3)`
- **Python 实现**：

```python
def var_p(values):
    return statistics.pvariance(values)

def var_s(values):
    return statistics.variance(values)

print(var_p([10, 20, 30]))
print(var_s([10, 20, 30]))
```

---

3. Python 实现文本函数

函数列表：`TEXT`、`LEFT`、`RIGHT`、`MID`、`LEN`、`TRIM`、`CONCAT`、`TEXTJOIN`、`SEARCH`、`FIND`

3.1 `TEXT`

- **描述**：按指定格式返回数值的文本表示。
- **Excel 示例**：

A	B
1234.56	=TEXT(A1, "0.00")

- **Excel 解决方案**：`=TEXT(A1, "0.00")`
- **Python 实现**：

```python
def text_format(value, format_spec):
    return format(value, format_spec)

print(text_format(1234.56, ".2f"))  # 输出: 1234.56
```

3.2 `LEFT`

- **描述**：返回文本开头的指定数量的字符。
- **Excel 示例**：

A	B
HelloWorld	=LEFT(A1, 5)

- **Excel 解决方案**：`=LEFT(A1, 5)`
- **Python 实现**：

```python
def left(text, num_chars):
    return text[:num_chars]

print(left("HelloWorld", 5))  # 输出: Hello
```

3.3 `RIGHT`

- **描述**：返回文本末尾的指定数量的字符。
- **Excel 示例**：

A	B
HelloWorld	=RIGHT(A1, 5)

- **Excel 解决方案**：`=RIGHT(A1, 5)`
- **Python 实现**：

```python
def right(text, num_chars):
    return text[-num_chars:]

print(right("HelloWorld", 5))  # 输出: World
```

3.4 `MID`

- **描述**：返回从文本中指定位置开始的指定数量的字符。
- **Excel 示例**：

A	B
HelloWorld	=MID(A1, 6, 5)

- **Excel 解决方案**：`=MID(A1, 6, 5)`
- **Python 实现**：

```python
def mid(text, start_pos, num_chars):
    return text[start_pos - 1:start_pos - 1 + num_chars]

print(mid("HelloWorld", 6, 5))  # 输出: World
```

3.5 `LEN`

- **描述**：返回文本的字符个数。
- **Excel 示例**：

A	B
HelloWorld	=LEN(A1)

- **Excel 解决方案**：`=LEN(A1)`
- **Python 实现**：

```python
def length(text):
    return len(text)

print(length("HelloWorld"))  # 输出：10
```

3.6 `TRIM`

- **描述**：去除文本中的多余空格（只保留单个空格）。
- **Excel 示例**：

A	B
Hello World	=TRIM(A1)

- **Excel 解决方案**：`=TRIM(A1)`
- **Python 实现**：

```python
def trim(text):
    return " ".join(text.split())

print(trim("  Hello   World   "))  # 输出：Hello World
```

3.7 `CONCAT`

- **描述**：将多个文本值合并为一个文本。
- **Excel 示例**：

A	B	C
Hello	World	=CONCAT(A1, " ", B1)

- **Excel 解决方案**：`=CONCAT(A1, " ", B1)`
- **Python 实现**：

```python
def concat(*args):
    return "".join(map(str, args))

print(concat("Hello", " ", "World"))  # 输出：Hello World
```

3.8 `TEXTJOIN`

- **描述**：合并文本并使用指定分隔符。
- **Excel 示例**：

A	B	C
Hello	World	=TEXTJOIN(" ", TRUE, A1, B1)

- **Excel 解决方案**：`=TEXTJOIN(" ", TRUE, A1, B1)`
- **Python 实现**：

```python
def textjoin(separator, *args):
    return separator.join(map(str, args))

print(textjoin(" ", "Hello", "World"))  # 输出: Hello World
```

3.9 `SEARCH`

- **描述**：返回文本中查找字符串的位置（不区分大小写）。
- **Excel 示例**：

A	B
HelloWorld	=SEARCH("World", A1)

- **Excel 解决方案**：`=SEARCH("World", A1)`
- **Python 实现**：

```python
def search(substring, text):
    return text.lower().find(substring.lower()) + 1

print(search("World", "HelloWorld"))  # 输出: 6
```

3.10 `FIND`

- **描述**：返回文本中查找字符串的位置（区分大小写）。
- **Excel 示例**：

A	B
HelloWorld	=FIND("World", A1)

- **Excel 解决方案**：`=FIND("World", A1)`
- **Python 实现**：

```python
def find(substring, text):
    try:
        return text.index(substring) + 1
    except ValueError:
        return -1

print(find("World", "HelloWorld"))  # 输出：6
```

---

4. Python 实现日期和时间函数

函数列表：`TODAY`、`NOW`、`DATE`、`YEAR`、`MONTH`、`DAY`、`HOUR`、`MINUTE`、`SECOND`、`DATEDIF`、`WEEKDAY`

4.1 `TODAY`

- **描述**：返回当前日期（无时间部分）。
- **Excel 示例**：

A
=TODAY()

- **Excel 解决方案**：`=TODAY()`
- **Python 实现**：

```python
from datetime import date

def today():
    return date.today()

print(today())  # 输出：当前日期（如 2025-01-17）
```

4.2 `NOW`

- **描述**：返回当前日期和时间。
- **Excel 示例**：

A
=NOW()

- **Excel 解决方案**：`=NOW()`
- **Python 实现**：

```python
from datetime import datetime

def now():
    return datetime.now()

print(now())  # 输出：当前日期和时间（如 2025-01-17 10:23:45）
```

4.3 `DATE`

- **描述**：返回由年、月和日组成的日期。
- **Excel 示例**：

A	B	C	D
年	月	日	=DATE(A1, B1, C1)
2025	1	17	

- **Excel 解决方案**：`=DATE(A1, B1, C1)`
- **Python 实现**：

```python
def create_date(year, month, day):
    return date(year, month, day)

print(create_date(2025, 1, 17))  # 输出：2025-01-17
```

4.4 `YEAR`，`MONTH`，`DAY`

- **描述**：
  - `YEAR`：返回日期的年份。
  - `MONTH`：返回日期的月份。
  - `DAY`：返回日期的天数。
- **Excel 示例**：

A	B	C	D
2025-01-17	=YEAR(A1)	=MONTH(A1)	=DAY(A1)

- **Excel 解决方案**：`=YEAR(A1)`，`=MONTH(A1)`，`=DAY(A1)`

- **Python 实现**：

```python
def get_year(date_obj):
    return date_obj.year

def get_month(date_obj):
    return date_obj.month

def get_day(date_obj):
    return date_obj.day

dt = date(2025, 1, 17)
print(get_year(dt))  # 输出: 2025
print(get_month(dt))  # 输出: 1
print(get_day(dt))  # 输出: 17
```

4.5 `HOUR`, `MINUTE`, `SECOND`

- **描述**：
  - `HOUR`：返回时间的小时部分。
  - `MINUTE`：返回时间的分钟部分。
  - `SECOND`：返回时间的秒数部分。
- **Excel 示例**：

A	B	C	D
10:23:45	=HOUR(A1)	=MINUTE(A1)	=SECOND(A1)

- **Excel 解决方案**：`=HOUR(A1)`，`=MINUTE(A1)`，`=SECOND(A1)`
- **Python 实现**：

```python
def get_hour(datetime_obj):
    return datetime_obj.hour

def get_minute(datetime_obj):
    return datetime_obj.minute

def get_second(datetime_obj):
    return datetime_obj.second

dt = datetime(2025, 1, 17, 10, 23, 45)
print(get_hour(dt))    # 输出: 10
print(get_minute(dt))  # 输出: 23
print(get_second(dt))  # 输出: 45
```

4.6 DATEDIF

- **描述**：返回两个日期之间的差异（以年、月或日表示）。
- **Excel 示例**：

A	B	C
2020-01-01	2025-01-01	=DATEDIF(A1, B1, "Y")

- **Excel 解决方案**：`=DATEDIF(A1, B1, "Y")`
- **Python 实现**：

```python
def datedif(start_date, end_date, unit):
    delta = end_date - start_date
    if unit == "Y":
        return end_date.year - start_date.year - ((end_date.month,
end_date.day) < (start_date.month, start_date.day))
    elif unit == "M":
        return (end_date.year - start_date.year) * 12 + end_date.month
- start_date.month - (end_date.day < start_date.day)
    elif unit == "D":
        return delta.days
    else:
        raise ValueError("Unsupported unit. Use 'Y', 'M', or 'D'.")

start = date(2020, 1, 1)
end = date(2025, 1, 1)
print(datedif(start, end, "Y"))  # 输出: 5
print(datedif(start, end, "M"))  # 输出: 60
print(datedif(start, end, "D"))  # 输出: 1826
```

4.7 WEEKDAY

- **描述**：返回日期的星期几（1 表示周日）。
- **Excel 示例**：

A	B
2025-01-17	=WEEKDAY(A1)

- **Excel 解决方案**：`=WEEKDAY(A1)`
- **Python 实现**：

```python
def weekday(date_obj):
    return date_obj.isoweekday() % 7 + 1

```

```
dt = date(2025, 1, 17)
print(weekday(dt))  # 输出：6（周五）
```

---

5. Python 实现查找与引用函数

函数列表：`VLOOKUP`、`HLOOKUP`、`INDEX`、`MATCH`、`XLOOKUP`、`OFFSET`

5.1 `VLOOKUP`

- **描述**：在表格中按列垂直查找，并返回对应行的值。
- **Excel 示例**：

A	B	C
产品ID	名称	=VLOOKUP(102, A1:B3, 2, FALSE)
101	苹果	
102	香蕉	
103	葡萄	

- **Excel 解决方案**：`=VLOOKUP(102, A1:B3, 2, FALSE)`
- **Python 实现**：

```
def vlookup(value, table, col_index):
    for row in table:
        if row[0] == value:
            return row[col_index - 1]
    return None

table = [
    [101, "苹果"],
    [102, "香蕉"],
    [103, "葡萄"]
]
print(vlookup(102, table, 2))  # 输出：香蕉
```

5.2 `HLOOKUP`

- **描述**：在表格中按行水平查找，并返回对应列的值。
- **Excel 示例**：

A	B	C
ID	101	102
名称	苹果	香蕉
=HLOOKUP(102, A1:C2, 2, FALSE)		

- **Excel 解决方案**：`=HLOOKUP(102, A1:C2, 2, FALSE)`
- **Python 实现**：

```python
def hlookup(value, table, row_index):
    header = table[0]
    for i, header_value in enumerate(header):
        if header_value == value:
            return table[row_index - 1][i]
    return None

table = [
    ["ID", 101, 102],
    ["名称", "苹果", "香蕉"]
]
print(hlookup(102, table, 2))  # 输出：香蕉
```

5.3 `INDEX`

- **描述**：按行列索引返回表格中的值。
- **Excel 示例**：

A	B
苹果	香蕉
葡萄	菠萝
=INDEX(A1:B2, 2, 1)	

- **Excel 解决方案**：`=INDEX(A1:B2, 2, 1)`
- **Python 实现**：

```python
def index(table, row_index, col_index):
    return table[row_index - 1][col_index - 1]

table = [
    ["苹果", "香蕉"],
    ["葡萄", "菠萝"]
]
```

```
print(index(table, 2, 1))  # 输出：葡萄
```

5.4 `MATCH`

- **描述**：返回值在表格中的位置（行或列索引）。
- **Excel 示例**：

A	B	C
苹果	香蕉	葡萄
=MATCH("香蕉", A1:C1, 0)		

- **Excel 解决方案**：`=MATCH("香蕉", A1:C1, 0)`
- **Python 实现**：

```
def match(value, array, match_type=0):
    if match_type == 0:  # 精确匹配
        for i, item in enumerate(array):
            if item == value:
                return i + 1
    return None

array = ["苹果", "香蕉", "葡萄"]
print(match("香蕉", array))  # 输出：2
```

5.5 `XLOOKUP`

- **描述**：按行或列查找值并返回对应的结果（支持双向查找）。
- **Excel 示例**：

A	B	C
ID	名称	价格
101	苹果	5
102	香蕉	3
=XLOOKUP(102, A2:A3, B2:B3)		

- **Excel 解决方案**：`=XLOOKUP(102, A2:A3, B2:B3)`
- **Python 实现**：

```python
def xlookup(value, lookup_array, return_array):
    for i, lookup_value in enumerate(lookup_array):
        if lookup_value == value:
            return return_array[i]
    return None

lookup_array = [101, 102, 103]
return_array = ["苹果", "香蕉", "葡萄"]
print(xlookup(102, lookup_array, return_array))  # 输出：香蕉
```

5.6 `OFFSET`

- **描述**：按偏移量返回指定单元格范围的值。
- **Excel 示例**：

A	B
苹果	香蕉
葡萄	菠萝
=OFFSET(A1, 1, 1)	

- **Excel 解决方案**：`=OFFSET(A1, 1, 1)`
- **Python 实现**：

```python
def offset(table, start_row, start_col, row_offset, col_offset):
    new_row = start_row + row_offset - 1
    new_col = start_col + col_offset - 1
    return table[new_row][new_col]

table = [
    ["苹果", "香蕉"],
    ["葡萄", "菠萝"]
]
print(offset(table, 1, 1, 1, 1))  # 输出：菠萝
```

---

6. 逻辑函数

函数列表：`IF`、`AND`、`OR`、`NOT`、`IFERROR`

6.1 IF

- **描述**：根据逻辑测试返回不同的值。
- **Excel 示例**：

A	B	C
50	及格	=IF(A1>=60, "及格", "不及格")

- **Excel 解决方案**：`=IF(A1>=60, "及格", "不及格")`
- **Python 实现**：

```python
def conditional_if(condition, value_if_true, value_if_false):
    return value_if_true if condition else value_if_false

print(conditional_if(50 >= 60, "及格", "不及格"))  # 输出：不及格
```

6.2 AND

- **描述**：当所有条件均为真时返回真。
- **Excel 示例**：

A	B	C
60	70	=AND(A1>=50, B1>=50)

- **Excel 解决方案**：`=AND(A1>=50, B1>=50)`
- **Python 实现**：

```python
def logical_and(*conditions):
    return all(conditions)

print(logical_and(60 >= 50, 70 >= 50))  # 输出：True
```

6.3 OR

- **描述**：当任一条件为真时返回真。
- **Excel 示例**：

A	B	C
40	70	=OR(A1>=50, B1>=50)

- **Excel 解决方案**：`=OR(A1>=50, B1>=50)`
- **Python 实现**：

```python
def logical_or(*conditions):
    return any(conditions)

print(logical_or(40 >= 50, 70 >= 50))  # 输出：True
```

6.4 `NOT`

- **描述**：返回逻辑值的否定。
- **Excel 示例**：

A	B
TRUE	=NOT(A1)

- **Excel 解决方案**：`=NOT(A1)`
- **Python 实现**：

```python
def logical_not(condition):
    return not condition

print(logical_not(True))  # 输出：False
```

6.5 `IFERROR`

- **描述**：如果表达式出错，返回指定的值。
- **Excel 示例**：

A	B
#DIV/0	=IFERROR(A1, "错误")

- **Excel 解决方案**：`=IFERROR(A1, "错误")`
- **Python 实现**：

```python
def if_error(expression, value_if_error):
    try:
        return expression()
    except Exception:
        return value_if_error

```

```
print(if_error(lambda: 1 / 0, "错误"))  # 输出：错误
```

---

7. 财务函数

函数列表：`PV`、`FV`、`PMT`、`NPER`、`RATE`

7.1 `PV` (现值)

- **描述**：计算未来值的当前价值。
- **Excel 示例**：

A	B	C	D
利率	期数	每期付款	=PV(A1, B1, C1)

- **Excel 解决方案**：`=PV(A1, B1, C1)`
- **Python 实现**：

```
def pv(rate, nper, pmt):
    return -pmt * ((1 - (1 + rate) ** -nper) / rate)

print(pv(0.05, 10, 100))  # 输出现值
```

7.2 `FV` (终值)

- **描述**：计算一笔投资在未来某个时间点的价值。
- **Excel 示例**：

A	B	C	D
利率	期数	每期付款	=FV(A1, B1, C1)

- **Excel 解决方案**：`=FV(A1, B1, C1)`
- **Python 实现**：

```
def fv(rate, nper, pmt):
    return -pmt * (((1 + rate) ** nper - 1) / rate)

print(fv(0.05, 10, 100))  # 输出终值
```

7.3 `PMT` (每期付款)

- **描述**：计算贷款的每期付款额。
- **Excel 示例**：

A	B	C	D
利率	期数	贷款总额	=PMT(A1, B1, C1)

- **Excel 解决方案**：`=PMT(A1, B1, C1)`
- **Python 实现**：

```python
def pmt(rate, nper, pv):
    return -pv * rate / (1 - (1 + rate) ** -nper)

print(pmt(0.05, 10, 1000))  # 输出每期付款
```

7.4 `NPER` (期数)

- **描述**：计算投资或贷款的总期数。
- **Excel 示例**：

A	B	C	D
利率	每期付款	贷款总额	=NPER(A1, B1, C1)

- **Excel 解决方案**：`=NPER(A1, B1, C1)`
- **Python 实现**：

```python
import math

def nper(rate, pmt, pv):
    return math.log(pmt / (pmt - rate * pv)) / math.log(1 + rate)

print(nper(0.05, 100, 1000))  # 输出期数
```

7.5 `RATE` (利率)

- **描述**：计算每期利率。
- **Excel 示例**：

A	B	C	D

期数	每期付款	贷款总额	=RATE(A1, B1, C1)

- **Excel 解决方案**：`=RATE(A1, B1, C1)`
- **Python 实现**：

```python
def rate(nper, pmt, pv):
    low, high = 0.0001, 1
    for _ in range(100):
        mid = (low + high) / 2
        if fv(mid, nper, pmt) + pv > 0:
            high = mid
        else:
            low = mid
    return mid

print(rate(10, 100, 1000))  # 输出利率
```

---

8. Python 实现信息函数

函数列表：`ISBLANK`、`ISERROR`、`TYPE`、`INFO`

8.1 `ISBLANK`

- **描述**：检查单元格是否为空。
- **Excel 示例**：

A	B
空白	=ISBLANK(A1)
123	

- **Excel 解决方案**：`=ISBLANK(A1)`
- **Python 实现**：

```python
def is_blank(value):
    return value is None or value == ""

print(is_blank(None))  # 输出: True
print(is_blank(123))   # 输出: False
```

8.2 ISERROR

- **描述**：检查公式是否返回错误。
- **Excel 示例**：

A	B
#DIV/0!	=ISERROR(A1)
123	

- **Excel 解决方案**：`=ISERROR(A1)`
- **Python 实现**：

```python
def is_error(expression):
    try:
        eval(expression)
        return False
    except Exception:
        return True

print(is_error("1/0"))  # 输出: True
print(is_error("1+1"))  # 输出: False
```

8.3 TYPE

- **描述**：返回值的数据类型（1=数字，2=文本，4=逻辑值，16=错误）。
- **Excel 示例**：

A	B
123	=TYPE(A1)
"Hello"	

- **Excel 解决方案**：`=TYPE(A1)`
- **Python 实现**：

```python
def value_type(value):
    if isinstance(value, (int, float)):
        return 1  # 数字
    elif isinstance(value, str):
        return 2  # 文本
    elif isinstance(value, bool):
        return 4  # 逻辑值
    elif isinstance(value, Exception):
        return 16  # 错误
```

```
    else:
        return 0  # 未知

print(value_type(123))       # 输出: 1
print(value_type("Hello"))   # 输出: 2
print(value_type(True))      # 输出: 4
```

8.4 `INFO`

- **描述**：返回关于当前操作环境的信息。
- **Excel 示例**：

A	B
系统信息	=INFO("system")

- **Excel 解决方案**：`=INFO("system")`
- **Python 实现**：

```
import platform

def get_info():
    return {
        "system": platform.system(),
        "version": platform.version(),
        "architecture": platform.architecture()[0]
    }

print(get_info())  # 输出系统信息
```

---

9. Python 实现数组函数

函数列表：`FILTER`、`UNIQUE`、`SORT`、`SEQUENCE`

9.1 `FILTER`

- **描述**：根据条件筛选数组。
- **Excel 示例**：

A	B	C
10	TRUE	=FILTER(A1:A3, B1:B3)

20	FALSE	
30	TRUE	

- **Excel 解决方案**：`=FILTER(A1:A3, B1:B3)`
- **Python 实现**：

```python
def filter_array(array, condition):
    return [val for val, cond in zip(array, condition) if cond]

print(filter_array([10, 20, 30], [True, False, True]))  # 输出：[10, 30]
```

9.2 `UNIQUE`

- **描述**：返回数组中的唯一值。
- **Excel 示例**：

A	B
10	=UNIQUE(A1:A5)
20	
10	
30	

- **Excel 解决方案**：`=UNIQUE(A1:A5)`
- **Python 实现**：

```python
def unique(array):
    return list(set(array))

print(unique([10, 20, 10, 30]))  # 输出：[10, 20, 30]
```

9.3 `SORT`

- **描述**：按升序或降序排序数组。
- **Excel 示例**：

A	B
30	=SORT(A1:A5, 1)

10	
20	

- **Excel 解决方案**：`=SORT(A1:A5, 1)`
- **Python 实现**：

```python
def sort_array(array, reverse=False):
    return sorted(array, reverse=reverse)

print(sort_array([30, 10, 20]))  # 输出：[10, 20, 30]
```

9.4 `SEQUENCE`

- **描述**：生成数字序列。
- **Excel 示例**：

A
=SEQUENCE(5, 1)

- **Excel 解决方案**：`=SEQUENCE(5, 1)`
- **Python 实现**：

```python
def sequence(n, step=1, start=1):
    return list(range(start, start + n * step, step))

print(sequence(5, step=1, start=1))  # 输出：[1, 2, 3, 4, 5]
```

---

10. Python 实现工程函数

函数列表：`CONVERT`、`DEC2BIN`、`HEX2DEC`

10.1 `CONVERT`

- **描述**：在不同单位间转换。
- **Excel 示例**：

A	B	C
1	m	=CONVERT(A1, "m", "cm")

- **Excel 解决方案**：`=CONVERT(A1, "m", "cm")`
- **Python 实现**：

```python
def convert(value, from_unit, to_unit):
    conversion = {"m": 1, "cm": 100}
    return value * (conversion[to_unit] / conversion[from_unit])

print(convert(1, "m", "cm"))  # 输出：100
```

10.2 `DEC2BIN`

- **描述**：将十进制数转换为二进制。
- **Excel 示例**：

A	B
10	=DEC2BIN(A1)

- **Excel 解决方案**：`=DEC2BIN(A1)`
- **Python 实现**：

```python
def dec2bin(value):
    return bin(value)[2:]

print(dec2bin(10))  # 输出：1010
```

10.3 `HEX2DEC`

- **描述**：将十六进制数转换为十进制。
- **Excel 示例**：

A	B
A	=HEX2DEC(A1)

- **Excel 解决方案**：`=HEX2DEC(A1)`
- **Python 实现**：

```python
def hex2dec(value):
    return int(value, 16)

print(hex2dec("A"))  # 输出：10
```